

1) USER INTERFACE DESIGN

- * User interface UI design is the process of designing the visual layout and interactive elements of a software application, website or digital system.
- * It focuses on how users interact with the system through screens, buttons, icons, menus, forms and other elements.
- * UI design ensure
 - The system looks attractive
 - The system is easy to use
 - The interaction is smooth and clear
 - The user can complete tasks efficiently.

UI is what the user sees and touches in digital products.

Example:

- * If you create a hall booking system in java (like your previous programs)
- * UI → Input fields, buttons, labels, output display.
- * UX → How easily a user can look a hall without confusion

Goals of UI Design

A goals UI should

- * Be simple
- * Be easy to understand
- * Be consistent
- * Reduce user events
- * Save user time
- * Be visually attractive

Principles of good UI Design

1) Clarity

- Everything should be clear and understandable
 - Clear button names (Submit, cancel)
 - Clear instructions.

2) Consistency

- Design elements should follow the same style
 - Same font style
 - Same button color
 - Same layout pattern

Example: If one page uses buttons, all pages should use blue buttons.

Simplicity

Avoid unnecessary elements

- Do not overload the screen
- Keep minimal options.

4) Feedback

* System should respond to user actions

Example:

- Login successful
- Invalid password
- Booking confirmed

* This tells the user what happened

5) Visibility

* Important features should be easy to find

- Search box at top
- Submit button at bottom of form

6) Error prevention

* Design should prevent mistakes

- Required field validation
- Password strength check
- Confirm before deleting

Element of UI Design

Input controls

Text boxes

Password fields

Radio buttons

check boxes

Dropdown menus.

1) Navigation components

Menu

Navigation box

Breadcrumbs

Tabs

2) Informational components:

Notifications

Progress bars

Message boxes

Tool tips

4) Containers

- * Cards
- * Accordions
- * Panels

Types of user interface

1) Graphical user interface (GUI)

→ Users windows, icons, buttons

Example

- Desktop application
- Android apps

2) Command line interface (CLI)

→ User types commands

→ Example : Terminal

3) Menu driven interface

→ User selects from menu

→ Example : ATM machine

Touch interface

- Used in mobile service
- Uses touch gestures

UI Design process

- Step 1 : Understand user requirements
- Step 2 : Create wireframe (basic layout sketch)
- Step 3 : Create mockup (visual design)
- Step 4 : Prototype (interactive sample)
- Step 5 : Testing

UI Design in software development.

In software projects

- * Requirement analysis
- * UI design
- * Coding
- * Testing
- * Deployment

UI is very important because :

- User judge the software within seconds
- Poor UI - User stop using the application

Example (Simple java program UI concepts)

Suppose you design a user registration system

Good UI:

Enter name :

Enter Email :

Enter Password :

Enter phone :

Bad UI :

Input 1

Input 2

Input 3

clear labels improve usability.

Advantages of good UI Design

* Improves user satisfaction

* Reduces training time

- * Reduce errors
- * Increases productivity
- * Increases business value

Disadvantage of poor UI Design

- * Confuses users
- * Increases mistakes
- * Wastes time
- * Reduces trust in systems

Conclusion

- * User interface design is a very important part of software development. It focuses on how the system looks and how users interact with it.
- * A good UI is simple, consistent, clean and user-friendly.

2. Symbolic execution

* Symbolic execution is a software testing and program analysis technique where a program is executed using symbolic inputs instead of actual input values.

* Instead of giving real values like

```
int x = 5;
```

* The program is analyzed for all possible input values at unit.

Symbolic execution unit

- * To detect bugs automatically
- * To find logical errors
- * To generate test cases
- * To check security vulnerabilities
- * To verify program correctness

How symbolic execution works

Lets take a simple example

```
if (  $x > 10$  ) {  
     $y = x + 5$ ;  
} else {  
     $y = x - 5$ ;  
}
```

Instead of assigning a real value to x , we treat x as Symbolic

Now the execution creates two paths:

Path 1

- * Condition : $x > 10$
- * Then : $y = x + 5$
- * Path constraint : $x > 10$

Path 2

- * Condition : $x \leq 10$
- * Then : $y = x - 5$
- * Path constraint : $x < 10$

So symbolic execution:

- * Explores all possible paths
- * Generates path conditions
- * check of constraints are valid.

Concepts in symbolic execution

- * Symbolic variables
- * Path constraints
- * Constraint solver

Model checking

* Model checking is a formal verification technique used to automatically verify whether a system satisfies certain properties.

* It checks:

→ "Does the system always behave correctly according to the specifications?"

Basic idea of model checking

Create a mathematical model of the system

Specify properties

Automatically check whether the model satisfies the properties.

If the property fails:

The tool gives a counterexample

Example of model checking

Suppose we have a traffic light system

technique
system

states:

- * Red
- * Green
- * Yellow

Property:

- * Red should never directly change to green without yellow.

* Model checking verifies

* Is this rule always satisfied?

* If not:

→ It gives the sequence of states that violate the rule.

Property

* Red should never directly change to green without yellow.

* Model checking verifies

* If this rule always satisfied

* If not:

→ It gives the sequence of states that violates the rule

Components of model checking

1. System model

Mathematical representation of system behavior

components of model checking

specification

Rules written in logical form. examples

* Safety property

* Liveness property

Model checker tool

software that verifies the model.

Types of properties checked

safety property

Something bad should not enter critical section simultaneously.

Liveness property

Something good should eventually happen.

Example: A request should eventually get a response.

Working process of model checking

- Step 1: Build system model
- Step 2: Write logical specification
- Step 3: Run model checker
- Step 4: If property fails - counter example generated

Advantage of model checking

- * Fully automatic
- * Gives counter examples
- * Detects design errors early
- * Very useful in hardware and protocol verification

3) Deployment tools

Deployment

- * Deployment is the process of making an application available for users after development and testing

- * It means moving your code from your local system to a system server on production environment so people can access it.

Example:

- * After developing a java web app → Deploy it to a server like apache tomcat
- * After developing a website - Deploy it to cloud platform like heroku on AWS

Deployment tools

- * Deployment tools help automate and manage the process of releasing software to server on cloud environments.

- * They help with:

→ uploading code

→ Building the project

→ Running tests

- Managing server
- Continuous integration, and survey.

Types of deployment tools:

a) Continuous integration / continuous deployment (CI) tools

- * These tools automatically build, test and deploy code when changes are made

Example:-

- Jenkins - open-source automation tool
- Github actions - CI/CD inside github
- Github CI/CD - Built-in devops tool.

Purpose:

- * Automates build and deployment
- * Reduces manual errors
- * Saves time

Configuration management tools

These tools manage server setup and environment configuration

Examples

- * Ansible
- * Puppet
- * Chef

Purpose

- * Automates server configuration
- * Ensures consistency across environment

Containerization tools

These tools package application with their dependencies

Examples

- * Docker
- * Kubernetes.

Purpose

- * Application runs the same everywhere
- * Easy scaling
- * Lightweight deployment

1) Cloud deployment platform

These platform allow hosting and deploying application online

Examples

- * Amazon web services
- * Microsoft azure
- * Google cloud platform

Purpose

- * Most applications
- * Most database
- * Provide security and scalability

Simple deployment workflow

Developer writes code

code pushed to git repository

CI tool builds and tests code

Deployment tool uploads code to server

Application goes live

Advantages of deployment tools

Faster releases

Reduced manual work

Better reliability

Easy rollback

Improved team collaboration

Example.

If you develop a java web application.

- * Build using maven
- * Deploy WAR file in apache tomcat
- * Use jenkins for automatic deployment

Conclusion

Deployment tools helps automate and manage the process of delivering software to users efficiently and safety.